



Snowflake × Salesforce

Integration Guide

Connecting Snowflake with Salesforce via
External Data Source & GPTfy API Data Source

Powered by



Table of Contents

1	Connecting Snowflake with Salesforce via External Data Source
1.1	Pre-Requisites
1.2	Snowflake Setup — Security Integration & OAuth
1.3	Snowflake Setup — Database, Tables & Data
1.4	Salesforce Setup — Auth Provider
1.5	Salesforce Setup — Named Credential (Legacy)
1.6	Salesforce Setup — External Data Source
1.7	Sync External Objects
1.8	Configure External Objects in Salesforce
1.9	Architecture Overview
2	Connecting Snowflake with Salesforce via GPTfy API Data Source
2.1	Pre-Requisites
2.2	Salesforce Setup — Auth Provider (SnowflakeDS)
2.3	Salesforce Setup — Named Credential (SnowflakeDS)
2.4	Apex Connector Class — SampleDataSourceClass2
2.5	GPTfy Setup — Create API Data Source
2.6	GPTfy Setup — Prompt with API Data Source Mapping
2.7	Comparison — External Data Source vs GPTfy API Data Source

Section 1 — Connecting Snowflake with Salesforce via External Data Source

This section provides a complete step-by-step walkthrough for establishing a live bi-directional connection between a **Snowflake** database and a **Salesforce** org using Salesforce's native **External Data Source** (Salesforce Connect — SQL type). Snowflake tables appear as **External Objects** inside Salesforce, enabling real-time data access without ETL pipelines or data duplication.

1.1 Pre-Requisites

- **Snowflake Account**

Admin-level access with the ability to create security integrations.

- **Salesforce Org with Connect**

Active Salesforce Connect licence (External Data Sources — SQL type).

- **GPTfy Installed**

GPTfy package installed in the Salesforce org.

- **Network Connectivity**

Salesforce IP ranges must be allowlisted in Snowflake's network policy.

1.2 Snowflake Setup — Security Integration & OAuth

Salesforce uses **OAuth 2.0** to authenticate against Snowflake's REST API. Create a **Security Integration** in Snowflake that registers Salesforce as a trusted OAuth client and specifies the callback URL.

SQL

```
-- Step 1: Create the OAuth Security Integration
Create OR replace SECURITY INTEGRATION MY_SNOWSQL_CLIENT_DEVORG
TYPE = OAUTH
ENABLED = TRUE
OAUTH_CLIENT = CUSTOM
OAUTH_CLIENT_TYPE = 'CONFIDENTIAL'
OAUTH_REDIRECT_URI = 'https://my.salesforce.com/services/authcallback/Snowflake'
OAUTH_ISSUE_REFRESH_TOKENS = TRUE
OAUTH_REFRESH_TOKEN_VALIDITY = 7776000
;

-- Step 2: Describe integration → retrieve Consumer Key & Endpoint URLs
DESC SECURITY INTEGRATION MY_SNOWSQL_CLIENT_DEVORG;

-- Step 3: Retrieve the Consumer Secret
select SYSTEM$SHOW_OAUTH_CLIENT_SECRETS( 'MY_SNOWSQL_CLIENT_DEVORG' );
```

1

Replace the integration name

Change MY_SNOWSQL_CLIENT_DEVORG in all three commands to a unique name for your environment.

2

Set redirect URI

Replace the placeholder domain with your Salesforce My Domain. The suffix /services/authcallback/Snowflake must match the Auth Provider URL Suffix.

3

Run lines 1–9

Select lines 1–9 and click Run. The security integration is created.

4

Run DESC command

Copy OAUTH_AUTHORIZATION_ENDPOINT, OAUTH_TOKEN_ENDPOINT, and OAUTH_CLIENT_ID from the output.

5

Run SHOW_SECRETS

Copy OAUTH_CLIENT_SECRET from the JSON output.

6

Update callback

After saving the Salesforce Auth Provider, Salesforce generates a callback URL. Update the OAUTH_REDIRECT_URI in Snowflake if it differs, then re-run steps 3–5.

1.3 Snowflake Setup — Database, Tables & Sample Data

Create the database, schema, and tables that will be surfaced as External Objects. The **EXTERNALID** column enables Salesforce record linkage.

SQL

```
-- 1. Database and schema
CREATE DATABASE TESTDB;
CREATE SCHEMA TESTDB.MYSCHEMA;

-- 2. Department table
CREATE OR REPLACE TABLE TESTDB.MYSCHEMA.DEPARTMENT (
  DID NUMBER(38,0) NOT NULL,
  DEPARTMENTNAME VARCHAR(25) NOT NULL,
  LOCATION VARCHAR(25),
  CREATEDDATE DATE DEFAULT CURRENT_DATE(),
  EXTERNALID VARCHAR(255),
  UNIQUE (DID)
```

SQL

```

);

-- 3. Employee table (FK → Department)
-- Note: DEPARTMENT_ID has no UNIQUE constraint – many employees can share a department
CREATE OR REPLACE TABLE TESTDB.MYSCHEMA.EMPLOYEE (
  EID NUMBER NOT NULL UNIQUE,
  EMPLOYEENAME VARCHAR(25) NOT NULL,
  EMAIL VARCHAR(25) DEFAULT NULL,
  JOBTITLE VARCHAR(25) DEFAULT NULL,
  DEPARTMENT_ID NUMBER NOT NULL,
  CREATEDDATE DATE DEFAULT CURRENT_DATE()
);
ALTER TABLE TESTDB.MYSCHEMA.EMPLOYEE
ADD FOREIGN KEY (DEPARTMENT_ID) REFERENCES TESTDB.MYSCHEMA.DEPARTMENT(DID);

-- 4. Seed data – Department
INSERT INTO TESTDB.MYSCHEMA.DEPARTMENT (DID, DEPARTMENTNAME, LOCATION) VALUES
(101,'Sales','Floor 1'),(102,'Marketing','Floor 3'),
(103,'Engineering','Floor 7'),(104,'Customer Service','Floor 2'),
(105,'Finance','Floor 4');
INSERT INTO TESTDB.MYSCHEMA.DEPARTMENT (DID,DEPARTMENTNAME,LOCATION,EXTERNALID)
VALUES (106,'Finance','Floor 5','0018d00000eE0CFAA0');

-- 5. Seed data – Employee
INSERT INTO TESTDB.MYSCHEMA.EMPLOYEE (EID,EMPLOYEENAME,EMAIL,JOBTITLE,DEPARTMENT_ID) VALUES
(201,'TestName1','testName1@test.com','Software Engineer',103),
(202,'TestName2','testName2@test.com','Marketing Manager',102),
(203,'TestName3','testName3@test.com','Finance',105),
(204,'TestName4','testName4@test.com','Customer Service',104),
(205,'TestName5','testName5@test.com','Software Engineer',103),
(206,'TestName6','testName6@test.com','Marketing Manager',102),
(207,'TestName7','testName7@test.com','Salesman',101);

-- 6. Verify
SELECT * FROM TESTDB.MYSCHEMA.DEPARTMENT;
SELECT t1.EID,t1.EMPLOYEENAME,t2.DEPARTMENTNAME
FROM TESTDB.MYSCHEMA.EMPLOYEE t1
INNER JOIN TESTDB.MYSCHEMA.DEPARTMENT t2 ON t1.DEPARTMENT_ID = t2.DID
LIMIT 100;
SELECT * FROM TESTDB.MYSCHEMA.DEPARTMENT WHERE EXTERNALID='0018d00000eE0CFAA0';

```

**NOTE**

The EXTERNALID column on DEPARTMENT stores the Salesforce record ID (e.g. an Account Id). This is the key linkage field — Salesforce queries filter by this value at runtime. Ensure DEPARTMENT_ID on EMPLOYEE has no UNIQUE constraint so multiple employees can belong to the same department.

1.4 Salesforce Setup — Auth Provider

The Auth Provider stores the OAuth 2.0 credentials from Snowflake and handles token exchange. **Navigation:** Setup → Identity → Auth. Providers → New

Provider Type	Open ID Connect
Name	Snowflake
URL Suffix	Snowflake (auto-filled)
Consumer Key	From DESC SECURITY INTEGRATION → OAUTH_CLIENT_ID
Consumer Secret	From SYSTEM\$SHOW_OAUTH_CLIENT_SECRETS
Authorize Endpoint URL	https://<account>.snowflakecomputing.com/oauth/authorize
Token Endpoint URL	https://<account>.snowflakecomputing.com/oauth/token-request
Use PKCE Extension	☐ Checked
Send access token in header	☐ Checked
Include Consumer Secret in SOAP API	☐ Checked

Screenshot — Auth Provider Configuration


Auth. Providers

Auth. Provider

Auth. Provider Detail
Edit Delete Clone

Auth. Provider ID	0SOJ7000000PB2j		
Provider Type	Open ID Connect		
Name	Snowflake		
URL Suffix	Snowflake		
Consumer Key	XmLPsgnNzEjkUJYnrK4tU9doCoM=		
Consumer Secret	EZjDXXVh26hKKc1OVallErZ7/B/Z8WCA7GC8ofY1RkU=		
	Copy		
Authorize Endpoint URL	https://vj79561.europe-west4.gcp.snowflakecomputing.com/oauth/authorize		
Token Endpoint URL	https://vj79561.europe-west4.gcp.snowflakecomputing.com/oauth/token-request		
User Info Endpoint URL			
Use Proof Key for Code Exchange (PKCE) Extension	☒	i	
Token Issuer			
Default Scopes			
Send access token in header	☒	i	Send client credentials
Include Consumer Secret in SOAP API Responses	☒	i	
Custom Error URL			
Custom Logout URL			
Registration Handler Type	None		
Portal			
Icon URL			
Use Salesforce MFA for this SSO Provider	☐	i	

Figure 1.4 — Salesforce Auth Provider 'Snowflake' (OpenID Connect)

**IMPORTANT**

After saving, Salesforce generates a Callback URL at the bottom of the Auth Provider page. Copy this URL and update your Snowflake Security Integration's OAUTH_REDIRECT_URI to match, then re-run DESC and SHOW_SECRETS to refresh the credentials.

1.5 Salesforce Setup — Named Credential (Legacy)

Stores the Snowflake REST API endpoint and references the Auth Provider for OAuth token management. Use the **Legacy** type. **Navigation:** Setup → Security → Named Credentials → New Legacy

Label	Snowflake
-------	-----------

Name	Snowflake
URL	https://<account>.snowflakecomputing.com/api/v2/statements
Identity Type	Named Principal
Authentication Protocol	OAuth 2.0
Authentication Provider	Snowflake (Auth Provider created above)
Generate Authorization Header	☐ Checked
Allow Merge Fields in HTTP Header	☐ Checked

Screenshot — Named Credential Configuration


SETUP
Named Credentials

Named Credential: Snowflake

Specify the callout endpoint's URL and the authentication settings that are required for Salesforce to make callouts to the remote system

« [Back to Named Credentials](#)

EditDelete

Label	Snowflake
Name	Snowflake
URL	https://vj79561.europe-west4.gcp.snowflakecomputing.com/api/v2/statements

▼ Authentication

Certificate	
Identity Type	Named Principal
Authentication Protocol	OAuth 2.0
Authentication Provider	Snowflake
Scope	
Authentication Status	Authenticated

▼ Callout Options

Generate Authorization Header	☒
Allow Merge Fields in HTTP Header	☒
Allow Merge Fields in HTTP Body	☐
Outbound Network Connection	

Figure 1.5 — Named Credential 'Snowflake' — OAuth 2.0 — Status: Authenticated

1.6 Salesforce Setup — External Data Source

The External Data Source is the Salesforce Connect configuration that maps Snowflake as a SQL-based external system. **Navigation:** Setup → Integrations → External Data Sources → New

External Data Source (Label)	testSnowflake
Name	testSnowflake
Type	SQL
Provider	Snowflake
Named Credential	Snowflake (created in Step 1.5)
Connection Timeout (Seconds)	120
Writable External Objects	☒ Checked — enables DML from Salesforce
Server Driven Pagination	☒ Checked — improves performance with large datasets

Screenshot — External Data Source Configuration

SETUP
External Data Sources

External Data Source Edit: testSnowflake

Connect to another Salesforce org or a third-party database or content system.

Save Save and New Cancel

External Data Source

Name

Type

Provider

Parameters

Named Credential

Connection Timeout (Seconds)

Writable External Objects ☒

Server Driven Pagination ☒

Save Save and New Cancel

Figure 1.6 — External Data Source 'testSnowflake' — SQL Type with Snowflake Provider

1.7 Sync External Objects

After saving the External Data Source, synchronise with Snowflake to import table metadata as External Objects.

1

Click 'Validate and Sync'

From the External Data Source detail page, click Validate and Sync.

2

Enter connection parameters

Database = TESTDB | Schema = MYSCHEMA

3

Select objects

Select DEPARTMENT and EMPLOYEE. Do NOT check 'Sync in Background' — run synchronously.

4

Verify deployment

Go to Setup → External Objects. If Not Deployed, click Edit on each and check the Deployed checkbox.

5

Create tabs

Create Salesforce App tabs for both External Objects via Setup → User Interface → Tabs → New.

6

Add related list

Open a Department record. If the Employee related list is missing, add it via Page Layouts in Setup.

**WARNING**

If sync fails with an authentication error, ensure the Named Credential status is 'Authenticated'. Click 'Authenticate' on the Named Credential to trigger the OAuth flow.

**NOTE**

After sync, External Objects are named with the '__x' suffix in Salesforce (e.g. DEPARTMENT__x, EMPLOYEE__x). These are virtual — no data is stored in Salesforce. Every SOQL query or page load fires a live callout to Snowflake via the Named Credential.

1.8 Configure External Objects in Salesforce

☐ **Record pages**

Use Lightning App Builder to add External Object related lists to standard object pages.

☐ **GPTfy prompts**

Create a GPTfy prompt on the Department External Object. Add Employee as a related object in prompt field mapping.

☐ **Run the prompt**

Navigate to a Department record → add GPTfy Console → run the prompt to verify end-to-end flow.

☐ **SOQL queries**

External Objects support SOQL (with limitations): `SELECT Id, DepartmentName__c FROM Department__x LIMIT 10`

☐ **DML operations**

Writable External Objects enabled: users can insert, update, delete Snowflake rows directly from Salesforce UI or Apex.

1.9 Architecture Overview

Layer	Component	Purpose
Snowflake	Security Integration (OAuth)	Issues access/refresh tokens to Salesforce

Snowflake	TESTDB.MYSCHEMA tables	Source data: DEPARTMENT, EMPLOYEE
Salesforce	Auth Provider (OpenID Connect)	Manages OAuth token lifecycle
Salesforce	Named Credential	Stores endpoint URL + auth reference
Salesforce	External Data Source (SQL/Snowflake)	Salesforce Connect adapter
Salesforce	External Objects (DEPARTMENT__x, EMPLOYEE__x)	Virtual SF objects backed by Snowflake

Section 2 — Connecting Snowflake with Salesforce via GPTfy API Data Source

GPTfy's **API Data Source** is an alternative integration pattern that bypasses Salesforce Connect entirely. Instead of syncing External Objects, GPTfy calls Snowflake's REST API directly at **prompt runtime** via an Apex connector class. This gives you full control over the SQL query, data shaping, and callout timing — with no Salesforce Connect licence required.



REUSE

The Snowflake database setup (TESTDB, MYSCHEMA, DEPARTMENT, EMPLOYEE and Security Integration) is IDENTICAL to Section 1. No additional Snowflake-side setup is needed. Start directly from Section 2.2.

2.1 Pre-Requisites

- **Section 1 Snowflake setup complete**
Security Integration, TESTDB, MYSCHEMA, DEPARTMENT & EMPLOYEE must exist.
- **GPTfy installed**
GPTfy package deployed and active in the Salesforce org.
- **Apex deploy access**
Developer Console or VS Code with SFDX to deploy the connector class.
- **No Salesforce Connect licence**
The API Data Source uses standard HTTP callouts — no SF Connect licence needed.

2.2 Salesforce Setup — Auth Provider (SnowflakeDS)

Follow the same steps as **Section 1.4**, but use **SnowflakeDS** as the name to keep this credential separate from the External Data Source one. **Navigation:** Setup → Identity → Auth. Providers → New

Provider Type	Open ID Connect
Name	SnowflakeDS
URL Suffix	SnowflakeDS (auto-filled)
Consumer Key	From DESC SECURITY INTEGRATION → OAUTH_CLIENT_ID
Consumer Secret	From SYSTEM\$SHOW_OAUTH_CLIENT_SECRETS
Authorize Endpoint URL	https://st81340.ap-southeast-1.snowflakecomputing.com/oauth/authorize
Token Endpoint URL	https://st81340.ap-southeast-1.snowflakecomputing.com/oauth/token-request
Use PKCE Extension	☐ Checked

Send access token in header	☐ Checked
Include Consumer Secret in SOAP API	☐ Checked

Screenshot — Auth Provider (SnowflakeDS)

Auth. Providers

Auth. Provider

Auth. Provider Detail Edit Delete Clone

Auth. Provider ID	0SOJ7000000PGDW
Provider Type	Open ID Connect
Name	SnowflakeDS
URL Suffix	SnowflakeDS
Consumer Key	t4p9WU8dAZyVf/6sGyzKSpdrig=
Consumer Secret	52DWL2rFHIp14mdDubM3fnBL1QsJfR9HTI5E5pJtnk= Copy
Authorize Endpoint URL	https://st81340.ap-southeast-1.snowflakecomputing.com/oauth/authorize
Token Endpoint URL	https://st81340.ap-southeast-1.snowflakecomputing.com/oauth/token-request
User Info Endpoint URL	
Use Proof Key for Code Exchange (PKCE) Extension	☒ i
Token Issuer	
Default Scopes	
Send access token in header	☒ i
Include Consumer Secret in SOAP API Responses	☒ i
Custom Error URL	
Custom Logout URL	
Registration Handler Type	None
Portal	
Icon URL	
Use Salesforce MFA for this SSO Provider	☐ i

Send client credentials in header ☐

Figure 2.2 — Auth Provider 'SnowflakeDS' — OpenID Connect — ap-southeast-1 Snowflake account



IMPORTANT

CRITICAL — The Snowflake Security Integration's OAUTH_REDIRECT_URI for this credential **MUST** end with /services/authcallback/SnowflakeDS (matching the URL Suffix 'SnowflakeDS'), NOT /Snowflake. These are two separate Auth Providers requiring two separate Security Integrations in Snowflake. Reusing the same integration will cause authentication failures.

2.3 Salesforce Setup — Named Credential (SnowflakeDS)

Navigation: Setup → Security → Named Credentials → New Legacy

Label	SnowflakeDS
Name	SnowflakeDS
URL	https://st81340.ap-southeast-1.snowflakecomputing.com/api/v2/statements
Identity Type	Named Principal
Authentication Protocol	OAuth 2.0
Authentication Provider	SnowflakeDS (Auth Provider created above)
Authentication Status	Authenticated
Start Auth Flow on Save	☐ Checked
Generate Authorization Header	☐ Checked
Allow Merge Fields in HTTP Header	☐ Unchecked
Allow Merge Fields in HTTP Body	☐ Unchecked



NOTE

The Named Credential name 'SnowflakeDS' is referenced inside the Apex class via callout:SnowflakeDS (resolved from dataSource.Named_Credential__c). Ensure names match exactly.

Screenshot — Named Credential (SnowflakeDS)

The screenshot shows the 'Named Credentials' setup page in Salesforce. The 'Label' and 'Name' fields are both 'SnowflakeDS'. The 'URL' field contains 'https://st81340.ap-southeast-1.snowflakecomputing.com/api/v2/statements'. Under the 'Authentication' section, 'Identity Type' is 'Named Principal', 'Authentication Protocol' is 'OAuth 2.0', 'Authentication Provider' is 'SnowflakeDS', and 'Start Authentication Flow on Save' is checked. The 'Authentication Status' is 'Authenticated'. Under the 'Callout Options' section, 'Generate Authorization Header' is checked, 'Allow Merge Fields in HTTP Header' and 'Allow Merge Fields in HTTP Body' are unchecked, and 'Outbound Network Connection' is empty.

Figure 2.3 — Named Credential 'SnowflakeDS' — OAuth 2.0 — Status: Authenticated

2.4 Apex Connector Class — SampleDataSourceClass2

This global Apex class implements GPTfy's **AIDataSourceInterface**. At prompt runtime GPTfy calls `getExternalData()`, which fires a POST to Snowflake's SQL REST API and returns the result set. **Deploy this class before creating the API Data Source.**

APEX

```
global class SampleDataSourceClass2 implements AIDataSourceInterface {

    global String getExternalData(AI_Data_Source__c dataSource, String extractedData) {

        // Deserialize the current Salesforce record context passed by GPTfy
        Map maps =
        (Map) JSON.deserializeUntyped(extractedData);
        System.debug('extractedData : ' + maps.get('Id'));
    }
}
```

APEX

```
HttpRequest req = new HttpRequest();
req.setEndpoint('callout:' + dataSource.Named_Credential__c);
req.setMethod('POST');
req.setHeader('Content-Type', 'application/json');
req.setHeader('Accept', 'application/json');
req.setTimeout(60000);

// Join Employee + Department filtered by current record's Salesforce Id
String sql =
'SELECT t1.EID, t1.EmployeeName, t1.Email, t2.did, t2.departmentname ' +
'FROM TESTDB.myschema.Employee AS t1 ' +
'INNER JOIN TESTDB.myschema.Department t2 ' +
' ON t1.department_id = t2.did ' +
'WHERE t2.ExternalId = \'' + maps.get('Id') + '\'';

Map body = new Map{
'statement' => sql, 'timeout' => 60,
'warehouse' => 'COMPUTE_WH',
'database' => 'TESTDB',
'schema' => 'MYSCHEMA',
'role' => 'SYSADMIN'
};
req.setBody(JSON.serialize(body));

Http http = new Http();
HttpResponse res = http.send(req);

if (res.getStatusCode() == 200) {
Map response =
(Map) JSON.deserializeUntyped(res.getBody());
Object o = response.get('data');
System.debug('Query Result: ' + o);
return o.toString();
} else {
System.debug('Error: ' + res.getStatusCode() + ' - ' + res.getBody());
return 'Error: ' + res.getStatusCode() + ' - ' + res.getBody();
}
}
}
```

Class key points:

Interface	AIDataSourceInterface — GPTfy's standard connector contract
extractedData	JSON of the current Salesforce record. GPTfy passes this automatically at runtime.
Named_Credential__c	Resolves to 'SnowflakeDS' at runtime — stores endpoint URL and OAuth token

Remote Site Settings	NOT required. Named Credentials automatically whitelist the endpoint for callouts.
SQL filter	Joins EMPLOYEE and DEPARTMENT filtered by DEPARTMENT.ExternalId = current SF record Id
Snowflake endpoint	POST /api/v2/statements — Snowflake SQL API v2, returns data as JSON array
Return value	The data array from Snowflake response, injected into the GPTfy prompt as context
Security note	The Id value used in SQL comes from Salesforce's own record Id (system-generated, 18-char). Safe for direct concatenation — not user-supplied input.

2.5 GPTfy Setup — Create API Data Source

Navigation: GPTfy → API Data Source → New

The GPTfy API Data Source catalogue lists all available connectors. Select **Snowflake API** or create a **Test API Data Source** for the custom Apex connector.

Screenshot — GPTfy API Data Source Catalogue

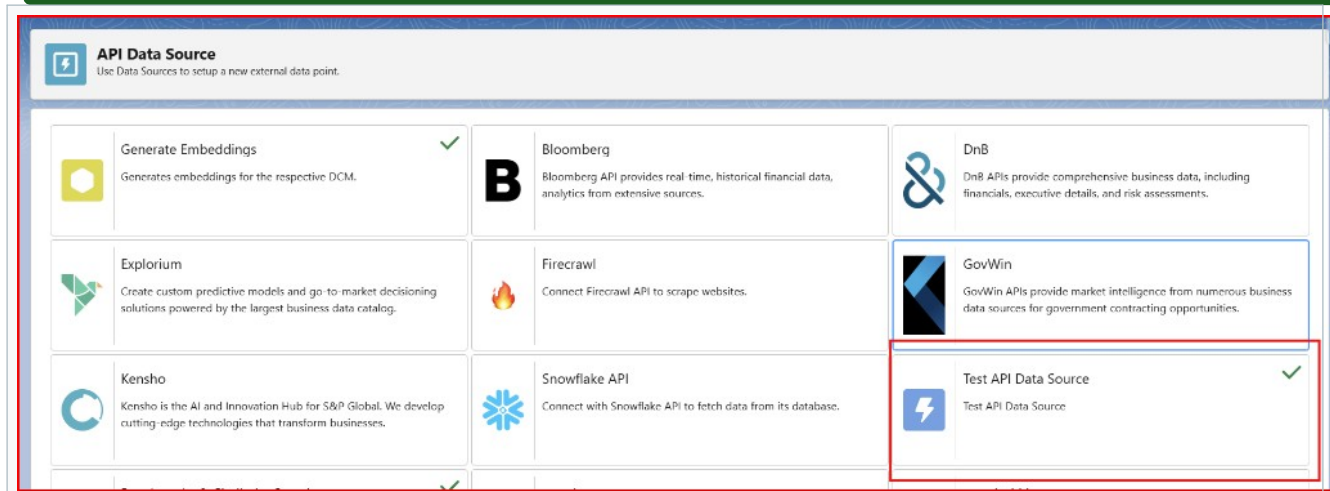


Figure 2.5 — GPTfy API Data Source catalogue — Snowflake API and Test API Data Source (active ☐)

Click **Edit** on your chosen data source and fill in the **Data Source Details** tab:

Data Source Name	Test API Data Source
Named Credential	SnowflakeDS
Source	Azure (GPTfy categorisation field only — does not affect Snowflake connectivity)
Connector Class	SampleDataSourceClass2 (the Apex class deployed in Step 2.4)
EndPoint URL	Leave blank — the endpoint is fully defined in the Named Credential URL

! IMPORTANT Click Save, then click Activate. The data source must show a green tick in the catalogue before it can be used in a prompt mapping.

Screenshot — Edit API Data Source

The screenshot shows a web application interface with a top navigation bar containing links like Cockpit, Prompt Catalog, ROI Dashboard, AI Insights Dashboard, Quality Insights, Trending, Prompts, Agents, Security Layers, Security Audits, Setup, and API Data Source. A modal dialog titled 'Edit Test API Data Source' is open. It has a close button (X) in the top right. The dialog contains two tabs: 'Info' and 'Data Source Details'. The 'Data Source Details' tab is selected. It has five input fields: 'Data Source Name' with the value 'Test API Data Source', 'Named Credential' with the value 'SnowflakeDS', 'Source' with a dropdown menu showing 'Azure', 'Connector Class' with a dropdown menu showing 'SampleDataSourceClass2', and 'EndPoint URL' which is empty. At the bottom of the dialog are four buttons: 'Cancel', 'Save', 'Delete', and 'Activate'.

Figure 2.5b — Edit Test API Data Source — Named Credential: SnowflakeDS, Connector Class: SampleDataSourceClass2

2.6 GPTfy Setup — Prompt with API Data Source Mapping

Link the active API Data Source to a prompt via **Data Context Mapping**. **Navigation:** GPTfy → Prompt Catalog → [Your Prompt] → Data Context Mapping → New

Mapping Name	Account Prompt Test ss 1 (or any descriptive name)
Target Object	Account (Account)
Target Object Label	Account
Select API Data Connection	Test API Data Source (active data source from Step 2.5)
Select Apex Security Layer	(optional)

Screenshot — Data Context Mapping

Data Context Mapping

* Mapping Name
Account Prompt Test ss 1

* Target Object ⓘ
Account (Account)

* Target Object Label ⓘ
Account

Description

Advanced Settings

Select API Data Connection
Test API Data Source

Select Apex Security Layer
Select Apex Security Layer

Cancel Save

Figure 2.6 — Data Context Mapping linking the prompt to 'Test API Data Source' on the Account object

Runtime flow:

1

User runs prompt

User opens an Account record in Salesforce and triggers the GPTfy prompt.

2

GPTfy serialises context

GPTfy serialises the Account record into JSON and calls `SampleDataSourceClass2.getExternalData()`.

3

Apex builds SQL

The class extracts the Account Id and builds a Snowflake SQL query filtering by DEPARTMENT.ExternalId.

4

Callout fires

HTTP POST fires to Snowflake /api/v2/statements via the SnowflakeDS Named Credential.

5

Data injected

Snowflake returns matching EMPLOYEE + DEPARTMENT rows. GPTfy injects the result into the prompt context for the LLM.

2.7 Comparison — External Data Source vs GPTfy API Data Source

Aspect	External Data Source (Sec 1)	GPTfy API Data Source (Sec 2)
SF Connect licence	Required	Not required
Data access	Always-on virtual objects (SOQL)	On-demand callout at prompt runtime
External Objects	Yes — visible in SF UI	No — data only in prompt output
DML from Salesforce	☐ Yes (writable ext. objects)	☐ No (read-only callout)
SQL control	Automatic (SF generates)	Full control via Apex class
Setup complexity	Moderate	Low (once Auth Provider/NC exist)
Best for	Browsing Snowflake data in SF	AI prompt enrichment with live data